

Orientar o agente com contexto

Prompts, instruções persistentes, skills, MCP e escolha de modelo.
Como comunicar ao agente o processo que definimos no Dia 1.

2H BASE → 7 BLOCOS → CONTEXTO > MODELO

Agenda do dia

7 blocos · 120 minutos (base) · até 180 min com demonstrações adicionais.

Bloco	Tema	Duração
1	Como agentes trabalham e por que erram	20 min
2	Prompts eficientes	22 min
3	Contexto persistente	22 min
4	Skills	20 min
5	MCP e segurança	10 min
6	Modelos e raciocínio	8 min
7	Exercício e encerramento	18 min
	Total	120 min

Contexto não informado não pode ser seguido

Um agente muito capaz ainda produz solução inadequada sem conhecer projeto, regras ou limites.



Caixas vazias ao redor do agente — cada uma é um contexto que ninguém informou.

Capacidade de modelo não substitui contexto.

Ciclo de trabalho do agente

Repetir até atender ao objetivo. Pedir só a implementação incentiva a pular as primeiras etapas.



Em mudanças maiores, peça explicitamente: analise, apresente um plano e só então implemente.

Por que agentes erram

Erros de agentes muitas vezes são erros de direcionamento, não de programação.

Objetivo vago

- “Melhore o projeto” não define objetivo.

Escopo aberto

- “Crie tudo o que for necessário.”

Contexto insuficiente

- Não conhece regras nem arquitetura.

Muitas tarefas juntas

- “Corrija todos os problemas.”

Permissão excessiva

- “Instale qualquer biblioteca.”

Ausência de validação

- “Está pronto?” aceita sem verificar.

Anatomia de um prompt eficiente

Não precisa ser sofisticado. Precisa ser completo — pode ser preenchido como um formulário.

1

Contexto

- Onde estamos e o que ler.

2

Objetivo

- Qual resultado único.

3

Escopo

- O que deve ser alterado.

4

Fora do escopo

- O que não alterar.

5

Restrições

- Bibliotecas, padrões, limites.

6

Critérios

- Como verificar.

7

Processo

- Analisar, planejar, implementar, revisar.

8

Validação

- Quais comandos executar.

Do pedido vago ao pedido executável

Quanto mais importante a decisão, menos ela deve ficar implícita.

ANTES · VAGO

"Faça um login moderno
para o sistema."

O que ficou indefinido? tecnologia, escopo, critérios, validação...

DEPOIS · ESTRUTURADO

Contexto: React organizado por features.

Objetivo: login simulado.

Escopo: form, validação, sessão, logout.

Fora do escopo: cadastro, backend, senha.

Restrições: não instalar deps.

Critérios: correta acessa; incorreta erra;
sessão permanece; logout encerra.

Processo: analise, planeje, implemente.

Validação: testes, lint e build.

Contexto temporário e contexto persistente

Nem toda instrução pertence ao prompt.

TEMPORÁRIO

Prompt da tarefa

- Objetivo específico
- Escopo atual
- Critérios atuais

PERSISTENTE

Arquivos do projeto

- Arquitetura e organização das pastas
- Comandos e padrões
- Limites arquiteturais e política de testes
- Definição de concluído e deps permitidas

AGENTS.md: contrato geral com agentes

Deve ser um contrato, não um manual enorme. Detalhes ficam em documentos ou skills.

Responde a

- O que ler antes de começar
- Como o projeto está organizado
- O que não pode ser feito
- Quais comandos executar
- Quando a tarefa pode ser concluída

EXEMPLO DE REGRA

Não instale dependências sem justificar.

Não altere funcionalidades fora do escopo.

Execute ``npm run validate`` antes de concluir.

CLAUDE.md: orientação específica da ferramenta

A regra geral fica no AGENTS.md. O CLAUDE.md é uma adaptação de uso — sem criar regras contraditórias.

Orienta o Claude Code a

- Ler o AGENTS.md
- Consultar documentos na ordem correta
- Localizar skills em .claude/skills
- Planejar mudanças maiores
- Executar os comandos do projeto

Alerta

- Se os arquivos repetirem grandes blocos, podem divergir com o tempo
- Prefira referências e instruções curtas

O que é uma skill

Um procedimento reutilizável: ensina ao agente como executar um tipo específico de tarefa.

Uma skill contém

- Quando usar
- Passos
- Regras
- Formato de saída

Exemplo: plan-feature

- Ler a arquitetura
- Identificar requisitos
- Listar arquivos afetados
- Dividir a execução e apontar riscos
- Não alterar código

O prompt diz o que precisamos agora. A skill descreve como executar um tipo de trabalho.

Anatomia e uso de uma skill

Uma skill útil é específica. Ruim: “ajude no projeto”. Melhor: “analise e gere requisitos, proposta, tarefas e riscos”.

ESTRUTURA MÍNIMA

```
plan-feature/  
├── SKILL.md  
└── references/
```

O SKILL.md traz:

```
name  
description  
instruções  
saída obrigatória
```

CATÁLOGO

Skills iniciais

- plan-feature · implement-feature
- review-changes · generate-tests
- update-documentation · prepare-pull-request

MCP: conexão com ferramentas externas

MCP entrega ferramentas e dados; não substitui regras de acesso.

Pode dar acesso a

- GitHub e arquivos
- Bancos de dados
- Documentação
- Ferramentas internas

Regras de segurança

- Servidores de origem conhecida
- Começar com leitura; menor acesso possível
- Nunca credenciais no prompt — usar variáveis de ambiente
- Confirmar operações destrutivas; remover acessos não usados

Modelo e raciocínio conforme a tarefa

Três perguntas: a tarefa é ambígua? um erro teria impacto? envolve muitas etapas ou ferramentas?

ALTO IMPACTO · BAIXA AMBIGUIDADE

Forte / alto raciocínio

- Mudança relevante, porém clara.

ALTO IMPACTO · ALTA AMBIGUIDADE

Forte + revisão humana

- Arquitetura, erro difícil.

BAIXO IMPACTO · BAIXA AMBIGUIDADE

Rápido / baixo raciocínio

- Renomear, formatar, documentar.

BAIXO IMPACTO · ALTA AMBIGUIDADE

Geral / médio

- Feature em vários arquivos.

Quanto mais respostas positivas, maior a capacidade e o nível de revisão. Raciocínio maior não corrige prompt incompleto.

Reescrever um prompt (em duplas)

Prompt inicial: “Adicione clima no sistema e deixe bonito.” — 7 min para reescrever.

PREENCHER

Campos de apoio

- Contexto
- Objetivo
- Escopo
- Não escopo
- Restrições
- Critérios
- Validação

VERSÃO DE REFERÊNCIA

Objetivo: clima atual por cidade.
Escopo: campo, busca, temperatura, condição, loading e erro.
Fora do escopo: previsão, geoloc, favoritos, design global.
Restrições: serviço do projeto; não instalar deps; não mexer em login/tarefas.
Processo: plano antes de alterar.
Validação: testes, lint, build.

Cinco camadas de orientação

- Prompt descreve a tarefa
- AGENTS.md registra as regras gerais
- CLAUDE.md orienta uma ferramenta específica
- Skills registram procedimentos
- MCP entrega ferramentas externas